

PHYS 449 - Homework 3, Question 1

Student Name: Allen Mathew

Student ID: 20964892

Due Date: Thursday, September 25, 3:00pm

ChatGPT was used to help generate the code

1. Produce a dataset $\mathcal{D}$ from a 8×8 Ising model. Each element of the dataset is a vector of spin configurations (drawn from the Boltzmann distribution at equilibrium at temperature T), and its respective classification of FM or PM . The dataset should consist of *i.i.d.* sampled configurations for T_i evenly distributed in the interval $[1.0, 3.5]$. Label each configuration vector sampled for $T < 2.269$ as FM , and those for $T > 2.269$ as PM . Build a data set of size $|\mathcal{D}| = 10^4$, and divide your dataset into 80% training and 20% validation.

The dataset, $\mathcal{D}$, of size 10^4 was created using the python code below, following what was asked in the question.

```
In [ ]: import numpy as np
        from tqdm import tqdm
        from MonteCarlo import init_spins, total_energy, energy_difference, metropolis_accept

        TC = 2.269          # critical temperature
        T_MIN, T_MAX = 1.0, 3.5
        L = 8                # lattice size for Q1(a)
        TOTAL_SAMPLES = 10_000
        TRAIN_FRAC = 0.80    # 80% training, 20% validation

        # Thinning to approximate i.i.d. samples
        BURN_IN_SWEEPS = 200
        THINNING_SWEEPS = 20

        def do_sweeps(spins, T, sweeps, J=1.0):
            L = spins.shape[0]
            N = L * L
            E = total_energy(spins, L, J)
            for _ in range(sweeps):
                for _ in range(N):
                    x, y = np.random.randint(0, L, size=2)
                    dE = energy_difference(spins, L, J, x, y)
                    if metropolis_accept(dE, T):
                        spins[x, y] *= -1
                        E += dE
            return E
```

```

def sample_iid_configs(L, T, n_samples, burn=BURN_IN_SWEEPS, thin=THINNING_SWEEPS,
    """Burn-in + thinning to approximate i.i.d. equilibrium samples at T."""
    spins = init_spins(L)
    _ = do_sweeps(spins, T, burn, J)
    cfgs = []
    for _ in range(n_samples):
        _ = do_sweeps(spins, T, thin, J)
        cfgs.append(spins.copy())
    return cfgs

def label_from_T(T, Tc=TC):
    # FM if T < Tc; PM if T > Tc (skip exact Tc)
    if T < Tc: return "FM"
    if T > Tc: return "PM"
    return None

def spins_to_vec(spins):
    return spins.reshape(-1).astype(np.int8) # {-1,+1} flattened

def build_dataset(L=8, total=10_000, Tmin=1.0, Tmax=3.5, n_temps=25, seed=42):
    np.random.seed(seed)
    Ts = np.linspace(Tmin, Tmax, n_temps)
    Ts = np.array([T for T in Ts if abs(T - TC) > 1e-8]) # avoid exact Tc boundary
    n_temps = len(Ts)
    per_T = total // n_temps
    leftover = total - per_T * n_temps

    X_list, y_list, T_list = [], [], []
    for i, T in enumerate(tqdm(Ts, desc="Temps for D")):
        n_here = per_T + (1 if i < leftover else 0)
        lbl = label_from_T(T)
        if lbl is None:
            continue
        cfgs = sample_iid_configs(L, T, n_here)
        for cfg in cfgs:
            X_list.append(spins_to_vec(cfg))
            y_list.append(lbl)
            T_list.append(T)

    X = np.vstack(X_list)
    y = np.array(y_list, dtype=object)
    Tarr = np.array(T_list, dtype=np.float32)

    # Shuffle & 80/20 split (train/val)
    idx = np.random.permutation(len(X))
    X, y, Tarr = X[idx], y[idx], Tarr[idx]

    n_train = int(round(TRAIN_FRAC * len(X)))
    X_train, y_train = X[:n_train], y[:n_train]
    X_val, y_val = X[n_train:], y[n_train:]

    print(f"L={L}, |D|={len(X)} -> train={len(X_train)}, val={len(X_val)}")
    return (X_train, y_train), (X_val, y_val), Tarr

# Building the dataset
(X_train, y_train), (X_val, y_val), T_used = build_dataset(

```

```

L=L, total=TOTAL_SAMPLES, Tmin=T_MIN, Tmax=T_MAX, n_temps=25, seed=123
)

# Save for reuse
np.savez_compressed("ising8_dataset_trainval.npz",
                    X_train=X_train, y_train=y_train, X_val=X_val, y_val=y_val, T=T)
print("Saved dataset to ising8_dataset_trainval.npz")

```

```

Temps for D: 100%|██████████| 25/25 [03:23<00:00, 8.15s/it]
L=8, |D|=10000 -> train=8000, val=2000
Saved dataset to ising8_dataset_trainval.npz

```

(a) Build a (hard) KNN classifier for the dataset, using the Hamming distance as your metric. Find the best value of the "hyperparameter" K (the closest training points) by optimizing the accuracy on the validation set.

To implement a hard k -nearest neighbour classifier for the dataset, we use the Hamming distance as our metric. It is given by the equation:

$$d(\vec{x}, \vec{x}') = \sum_{i=1}^N (1 - \delta_{x_i x'_i})$$

where $\delta_{x_i x'_i} = 1$ if $x_i = x'_i$ and 0 otherwise.

The KNN classifier looks at the K closest training samples based on this Hamming distance. In order to find the best value of K , we test multiple values of K and check which one gets the highest accuracy in the validation set.

We implement all this in the code below:

```

In [ ]: import numpy as np
        from tqdm import tqdm
        import matplotlib.pyplot as plt

        def knn_predict_hard_hamming(X_train, y_train, X_query, K=5):
            """
            Pure NumPy KNN (hard vote) with Hamming distance on {-1,+1} vectors.
            Hamming = fraction of differing components.
            """
            preds = []
            for x in tqdm(X_query, desc=f"KNN(K={K})"):
                # vectorized Hamming distances to all training samples
                dists = np.mean(X_train != x, axis=1)
                nn_idx = np.argpartition(dists, K)[:K]
                nn_labels = y_train[nn_idx]

                fm = np.sum(nn_labels == "FM")
                pm = K - fm
                if fm > pm:
                    preds.append("FM")
                elif pm > fm:
                    preds.append("PM")

```

```

        else:
            # tie-breaker: nearest neighbor label
            preds.append(y_train[nn_idx[np.argmin(dists[nn_idx])]])
    return np.array(preds, dtype=object)

def accuracy(y_true, y_pred):
    return float(np.mean(y_true == y_pred))

K_GRID = list(range(1, 51))

val_results = []
best = {"K": None, "acc": -1.0}
for K in K_GRID:
    y_hat = knn_predict_hard_hamming(X_train, y_train, X_val, K=K)
    acc = accuracy(y_val, y_hat)
    val_results.append((K, acc))
    if acc > best["acc"]:
        best = {"K": K, "acc": acc}

print("Validation accuracy by K:")
for K, acc in val_results:
    print(f"K={K:>2d}: acc={acc:.4f}")
print(f"\nBest K = {best['K']} with validation accuracy = {best['acc']:.4f}")

# Plot K vs accuracy
plt.figure(figsize=(6,4))
plt.plot([k for k,_ in val_results], [a for _,a in val_results], marker='o')
plt.xlabel("K")
plt.ylabel("Validation Accuracy")
plt.title("KNN (Hamming) on 8x8 Ising dataset")
plt.grid(True, alpha=0.4)
plt.show()

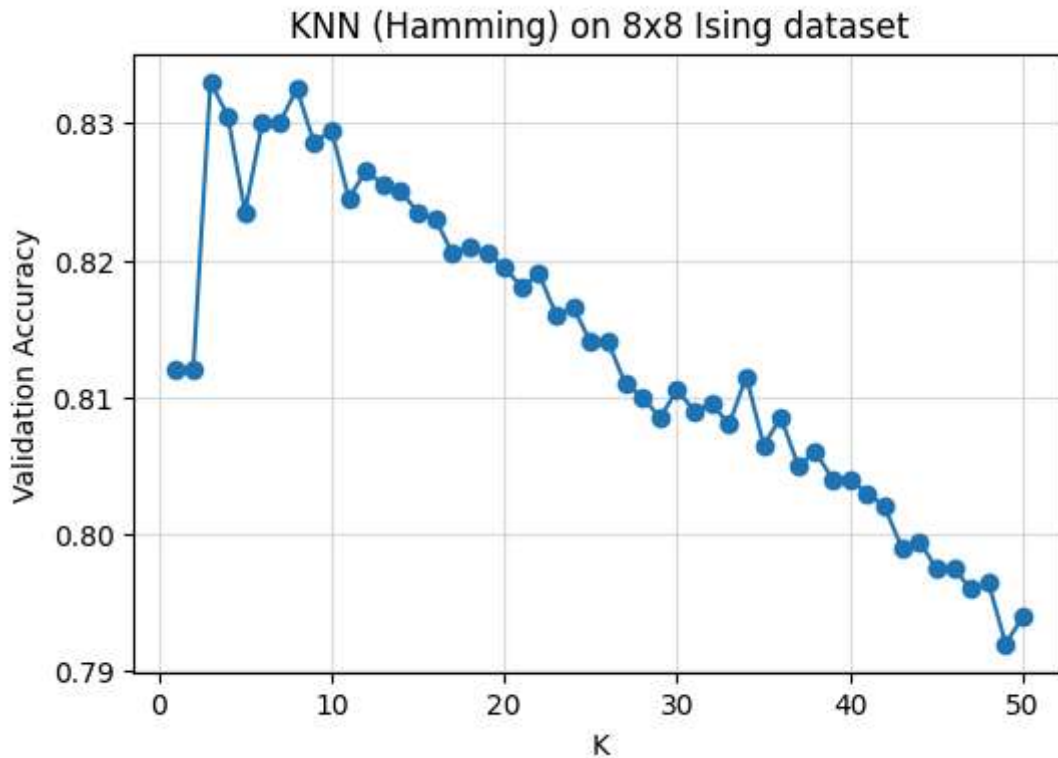
```

KNN(K=1): 100%		2000/2000	[00:00<00:00, 2002.11it/s]
KNN(K=2): 100%		2000/2000	[00:00<00:00, 2050.33it/s]
KNN(K=3): 100%		2000/2000	[00:00<00:00, 2046.40it/s]
KNN(K=4): 100%		2000/2000	[00:00<00:00, 2066.98it/s]
KNN(K=5): 100%		2000/2000	[00:00<00:00, 2065.44it/s]
KNN(K=6): 100%		2000/2000	[00:00<00:00, 2055.42it/s]
KNN(K=7): 100%		2000/2000	[00:00<00:00, 2010.37it/s]
KNN(K=8): 100%		2000/2000	[00:00<00:00, 2058.17it/s]
KNN(K=9): 100%		2000/2000	[00:00<00:00, 2055.11it/s]
KNN(K=10): 100%		2000/2000	[00:00<00:00, 2039.06it/s]
KNN(K=11): 100%		2000/2000	[00:00<00:00, 2029.24it/s]
KNN(K=12): 100%		2000/2000	[00:00<00:00, 2027.38it/s]
KNN(K=13): 100%		2000/2000	[00:00<00:00, 2018.05it/s]
KNN(K=14): 100%		2000/2000	[00:01<00:00, 1927.82it/s]
KNN(K=15): 100%		2000/2000	[00:01<00:00, 1775.70it/s]
KNN(K=16): 100%		2000/2000	[00:01<00:00, 1980.33it/s]
KNN(K=17): 100%		2000/2000	[00:00<00:00, 2035.07it/s]
KNN(K=18): 100%		2000/2000	[00:00<00:00, 2031.81it/s]
KNN(K=19): 100%		2000/2000	[00:00<00:00, 2014.48it/s]
KNN(K=20): 100%		2000/2000	[00:00<00:00, 2039.37it/s]
KNN(K=21): 100%		2000/2000	[00:00<00:00, 2029.45it/s]
KNN(K=22): 100%		2000/2000	[00:00<00:00, 2025.15it/s]
KNN(K=23): 100%		2000/2000	[00:00<00:00, 2003.80it/s]
KNN(K=24): 100%		2000/2000	[00:00<00:00, 2040.86it/s]
KNN(K=25): 100%		2000/2000	[00:00<00:00, 2066.50it/s]
KNN(K=26): 100%		2000/2000	[00:00<00:00, 2069.83it/s]
KNN(K=27): 100%		2000/2000	[00:00<00:00, 2046.77it/s]
KNN(K=28): 100%		2000/2000	[00:00<00:00, 2053.86it/s]
KNN(K=29): 100%		2000/2000	[00:00<00:00, 2027.98it/s]
KNN(K=30): 100%		2000/2000	[00:01<00:00, 1996.21it/s]
KNN(K=31): 100%		2000/2000	[00:00<00:00, 2021.01it/s]
KNN(K=32): 100%		2000/2000	[00:00<00:00, 2012.90it/s]
KNN(K=33): 100%		2000/2000	[00:00<00:00, 2037.94it/s]
KNN(K=34): 100%		2000/2000	[00:00<00:00, 2021.15it/s]
KNN(K=35): 100%		2000/2000	[00:00<00:00, 2057.25it/s]
KNN(K=36): 100%		2000/2000	[00:00<00:00, 2066.45it/s]
KNN(K=37): 100%		2000/2000	[00:00<00:00, 2027.39it/s]
KNN(K=38): 100%		2000/2000	[00:00<00:00, 2044.16it/s]
KNN(K=39): 100%		2000/2000	[00:00<00:00, 2032.21it/s]
KNN(K=40): 100%		2000/2000	[00:00<00:00, 2043.59it/s]
KNN(K=41): 100%		2000/2000	[00:00<00:00, 2020.73it/s]
KNN(K=42): 100%		2000/2000	[00:00<00:00, 2045.33it/s]
KNN(K=43): 100%		2000/2000	[00:00<00:00, 2046.60it/s]
KNN(K=44): 100%		2000/2000	[00:00<00:00, 2042.93it/s]
KNN(K=45): 100%		2000/2000	[00:00<00:00, 2062.99it/s]
KNN(K=46): 100%		2000/2000	[00:00<00:00, 2038.07it/s]
KNN(K=47): 100%		2000/2000	[00:00<00:00, 2062.52it/s]
KNN(K=48): 100%		2000/2000	[00:00<00:00, 2063.59it/s]
KNN(K=49): 100%		2000/2000	[00:00<00:00, 2062.82it/s]
KNN(K=50): 100%		2000/2000	[00:00<00:00, 2046.62it/s]

Validation accuracy by K:

K= 1: acc=0.8120
K= 2: acc=0.8120
K= 3: acc=0.8330
K= 4: acc=0.8305
K= 5: acc=0.8235
K= 6: acc=0.8300
K= 7: acc=0.8300
K= 8: acc=0.8325
K= 9: acc=0.8285
K=10: acc=0.8295
K=11: acc=0.8245
K=12: acc=0.8265
K=13: acc=0.8255
K=14: acc=0.8250
K=15: acc=0.8235
K=16: acc=0.8230
K=17: acc=0.8205
K=18: acc=0.8210
K=19: acc=0.8205
K=20: acc=0.8195
K=21: acc=0.8180
K=22: acc=0.8190
K=23: acc=0.8160
K=24: acc=0.8165
K=25: acc=0.8140
K=26: acc=0.8140
K=27: acc=0.8110
K=28: acc=0.8100
K=29: acc=0.8085
K=30: acc=0.8105
K=31: acc=0.8090
K=32: acc=0.8095
K=33: acc=0.8080
K=34: acc=0.8115
K=35: acc=0.8065
K=36: acc=0.8085
K=37: acc=0.8050
K=38: acc=0.8060
K=39: acc=0.8040
K=40: acc=0.8040
K=41: acc=0.8030
K=42: acc=0.8020
K=43: acc=0.7990
K=44: acc=0.7995
K=45: acc=0.7975
K=46: acc=0.7975
K=47: acc=0.7960
K=48: acc=0.7965
K=49: acc=0.7920
K=50: acc=0.7940

Best K = 3 with validation accuracy = 0.8330



We see that we got a peak 83.3% accuracy with $K = 3$. It's interesting to see the graph starting with an increasing accuracy trend as K increases but after $K = 3$, we see a slow decline in the validation accuracy.

(b) Increase both your dataset size and the size of the lattice. What is the largest lattice size that you can get reasonable performance of your KNN classifier on, for your own compute availability?

I did a total of 6 tests to see how dataset size and lattice size affected validation accuracy and computation time.

- $L = 8, |\mathcal{D}| = 10^4$
- $L = 8, |\mathcal{D}| = 1.5 \times 10^4$
- $L = 12, |\mathcal{D}| = 10^4$
- $L = 12, |\mathcal{D}| = 1.5 \times 10^4$
- $L = 16, |\mathcal{D}| = 10^4$
- $L = 16, |\mathcal{D}| = 1.5 \times 10^4$

These tests were ran using the code below:

```
In [ ]: import numpy as np
import matplotlib.pyplot as plt
from tqdm import tqdm

def run_scaling_experiment(
    L_list=(8, 12, 16),
    total_list=(10_000, 15_000, 20_000),
```

```

n_temps=25,
K_eval=(5, 7, 9),
train_frac=0.80,
seed=123
):
    results = [] # rows of (L, total, bestK, bestAcc)

    for L in L_list:
        for total in total_list:
            print(f"\n=== L={L}, |D|={total} ===")
            (X_tr, y_tr), (X_va, y_va), _ = build_dataset(
                L=L, total=total, Tmin=T_MIN, Tmax=T_MAX, n_temps=n_temps, seed=seed
            )

            best = {"K": None, "acc": -1.0}
            for K in K_eval:
                y_hat = knn_predict_hard_hamming(X_tr, y_tr, X_va, K=K)
                acc = accuracy(y_va, y_hat)
                if acc > best["acc"]:
                    best = {"K": K, "acc": acc}

            print(f"Best K={best['K']}, val acc={best['acc']:.4f}")
            results.append((L, total, best["K"], best["acc"]))

    return results

results = run_scaling_experiment(
    L_list=(8, 12, 16),
    total_list=(10_000, 15_000),
    K_eval=(5, 7, 9)
)

# Summarize and plot "best acc vs L"
Ls = sorted(set([r[0] for r in results]))
best_acc_by_L = {L: max([r[3] for r in results if r[0] == L]) for L in Ls}

plt.figure(figsize=(6,4))
plt.plot(Ls, [best_acc_by_L[L] for L in Ls], marker='o')
plt.xlabel("Lattice size L")
plt.ylabel("Best validation accuracy")
plt.title("KNN performance vs lattice size")
plt.grid(True, alpha=0.4)
plt.show()

print("\nSummary (L, |D|, best K, best acc):")
for row in results:
    print(row)

# Heuristic: pick "largest reasonable L" where val acc remains decent (e.g., ≥ ~0.8)
reasonable = [L for L in Ls if best_acc_by_L[L] >= 0.80]
if len(reasonable):
    print(f"\nLargest L with 'reasonable' (≥0.80) accuracy: L = {max(reasonable)}")
else:
    print("\nNo L met the 'reasonable' threshold; consider increasing |D| or adjust

```

```

=== L=8, |D|=10000 ===

```

Temps for D: 100%|██████████| 25/25 [03:21<00:00, 8.06s/it]

L=8, |D|=10000 -> train=8000, val=2000

KNN(K=5): 100%|██████████| 2000/2000 [00:01<00:00, 1269.57it/s]

KNN(K=7): 100%|██████████| 2000/2000 [00:01<00:00, 1304.42it/s]

KNN(K=9): 100%|██████████| 2000/2000 [00:01<00:00, 1332.92it/s]

Best K=9, val acc=0.8310

=== L=8, |D|=15000 ===

Temps for D: 100%|██████████| 25/25 [04:54<00:00, 11.78s/it]

L=8, |D|=15000 -> train=12000, val=3000

KNN(K=5): 100%|██████████| 3000/3000 [00:03<00:00, 850.89it/s]

KNN(K=7): 100%|██████████| 3000/3000 [00:03<00:00, 868.23it/s]

KNN(K=9): 100%|██████████| 3000/3000 [00:03<00:00, 890.14it/s]

Best K=7, val acc=0.8370

=== L=12, |D|=10000 ===

Temps for D: 100%|██████████| 25/25 [07:27<00:00, 17.91s/it]

L=12, |D|=10000 -> train=8000, val=2000

KNN(K=5): 100%|██████████| 2000/2000 [00:03<00:00, 531.01it/s]

KNN(K=7): 100%|██████████| 2000/2000 [00:03<00:00, 547.72it/s]

KNN(K=9): 100%|██████████| 2000/2000 [00:03<00:00, 580.73it/s]

Best K=5, val acc=0.8265

=== L=12, |D|=15000 ===

Temps for D: 100%|██████████| 25/25 [10:57<00:00, 26.29s/it]

L=12, |D|=15000 -> train=12000, val=3000

KNN(K=5): 100%|██████████| 3000/3000 [00:11<00:00, 265.73it/s]

KNN(K=7): 100%|██████████| 3000/3000 [00:08<00:00, 337.73it/s]

KNN(K=9): 100%|██████████| 3000/3000 [00:07<00:00, 394.85it/s]

Best K=5, val acc=0.8557

=== L=16, |D|=10000 ===

Temps for D: 100%|██████████| 25/25 [12:59<00:00, 31.18s/it]

L=16, |D|=10000 -> train=8000, val=2000

KNN(K=5): 100%|██████████| 2000/2000 [00:05<00:00, 372.63it/s]

KNN(K=7): 100%|██████████| 2000/2000 [00:05<00:00, 370.45it/s]

KNN(K=9): 100%|██████████| 2000/2000 [00:05<00:00, 384.83it/s]

Best K=5, val acc=0.8355

=== L=16, |D|=15000 ===

Temps for D: 100%|██████████| 25/25 [18:59<00:00, 45.59s/it]

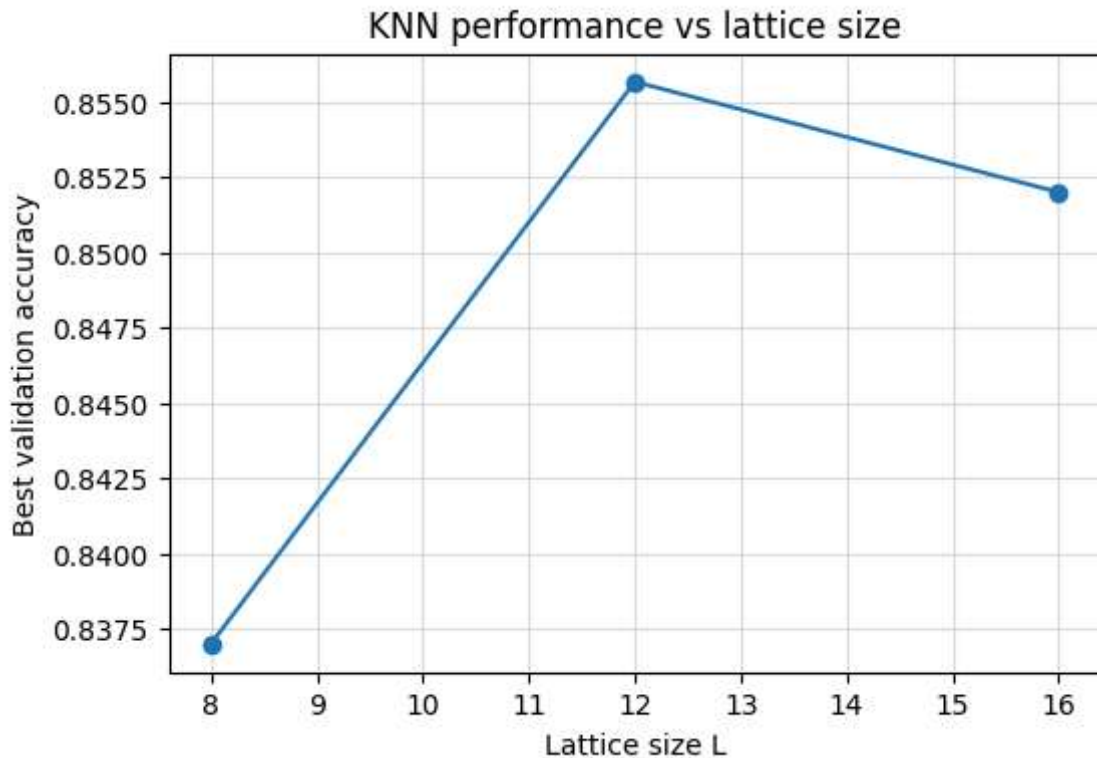
L=16, |D|=15000 -> train=12000, val=3000

KNN(K=5): 100%|██████████| 3000/3000 [00:13<00:00, 227.90it/s]

KNN(K=7): 100%|██████████| 3000/3000 [00:11<00:00, 254.81it/s]

KNN(K=9): 100%|██████████| 3000/3000 [00:11<00:00, 252.54it/s]

Best K=5, val acc=0.8520



Summary (L , $|\mathcal{D}|$, best K , best acc):

(8, 10000, 9, 0.831)
 (8, 15000, 7, 0.837)
 (12, 10000, 5, 0.8265)
 (12, 15000, 5, 0.8556666666666667)
 (16, 10000, 5, 0.8355)
 (16, 15000, 5, 0.852)

Largest L with 'reasonable' (≥ 0.80) accuracy: $L = 16$

From the test results, we find the following patterns:

- Increasing L greatly increases the runtime (3:21 minutes $\rightarrow$ 7:27 minutes from $L = 8$ to $L = 12$)
- Increasing L does not change the validation accuracy much, which makes sense since the dataset size is unchanged
- Increasing $|\mathcal{D}|$ increases the runtime, not as much as increasing L (3:21 minutes $\rightarrow$ 4:54 minutes from $|\mathcal{D}| = 10^4$ to $|\mathcal{D}| = 1.5 \times 10^4$)
- Increasing $|\mathcal{D}|$ increases the validation accuracy (82.65% $\rightarrow$ 85.57% in the $L = 12$ case)

It's important to note that for every time we increase L , the N in our Ising model changes by a factor of L^2 . This would explain why increasing L increases the computation time by so much.

Increasing L and $|\mathcal{D}|$ does increase validation accuracy, but the computation time increases significantly as you go to higher L and $|\mathcal{D}|$. I believe that $L \approx 12$ to 16 so that the user doesn't get bored waiting and still get a decently high accuracy.